Q1. Error Analysis of an Online Examination System

Aim

To identify and analyze errors in student registration, attendance eligibility, and examination access using Selenium WebDriver and black-box testing techniques.

a) Error Identification and Analysis

Error	Expected Behaviour	Actual Behaviour	Possible Cause
Attendance validation	Students below 75% should be rejected	Ineligible students can access exams	Incorrect attendance condition
Eligible student access	Students with 75% or more should be allowed	Eligible students are rejected	Incorrect comparison or calculation
Registration	Valid details should be accepted	Valid registration fails	Input validation error
Login	Valid credentials should allow login	Login fails occasionally	Authentication or session error
Exam access	Only eligible students should enter	Unauthorized access is possible	Missing eligibility check

Root cause: Incorrect conditions, improper validation, and missing access-control checks may cause these errors.

Correction: Validate registration details and check attendance eligibility on the server before granting examination access.

b) Black-Box Testing Techniques

1. Equivalence Class Partitioning

Divide attendance values into valid and invalid groups.

Input	Expected Result
50%	Rejected
74%	Rejected
75%	Accepted
80%	Accepted
110%	Invalid attendance

2. Boundary Value Analysis

Test values around the 75% boundary.

Attendance	Expected Result
74%	Rejected
74.9%	Rejected
75%	Accepted
75.1%	Accepted
76%	Accepted

Assumption: Attendance eligibility is based on an exact threshold of 75%.

3. Random Testing

Enter randomly selected attendance values, registration numbers, and login details to identify unexpected errors.

c) Decision Table and State Transition Testing

Valid Login	Attendance ≥ 75%	Expected Result
Yes	Yes	Exam access allowed
Yes	No	Exam access denied
No	Yes	Login rejected
No	No	Login rejected

State Transition Diagram

Registered

|

Login

|

Authenticated

|

Check Attendance

|

+--+---+

| |

Eligible Ineligible

| |

Exam Access

Access Denied

|

Submit Exam

|

Completed

d) Selenium WebDriver Testing

Example Python code:

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("http://localhost:5000/login")

driver.find_element(By.ID, "username").send_keys("student")
driver.find_element(By.ID, "password").send_keys("1234")
driver.find_element(By.ID, "login").click()

print("Login test executed")
driver.save_screenshot("exam_test.png")

driver.quit()
```

Note: This example assumes a local test application with the specified URL and element IDs. Additional checks are needed to verify attendance eligibility and exam access.

When a test fails, capture a screenshot and record the expected result, actual result, and defect ID.

e) Test Adequacy Evaluation

Test adequacy measures how thoroughly the system has been tested.

Formula:

Test Execution Percentage = (Executed Test Cases / Total Planned Test Cases) × 100

For example, if 8 out of 10 test cases are executed:

$$\frac{8}{10} \times 100 = 80\%$$

High-priority tests include attendance validation, unauthorized exam access, and eligible student access.

Conclusion: Black-box testing and Selenium help identify registration, attendance, and examination access errors. Additional tests should verify boundary values and unauthorized access before release.

Q2. Error Analysis of a Student Grade Calculator

Aim

To identify errors in a Java grade calculator and improve its correctness using JUnit, Maven, and JaCoCo.

a) Error Identification

Common errors include:

1. Incorrect grade conditions.
2. Missing validation for marks below 0 or above 100.
3. Incorrect handling of boundary marks.
4. Incorrect use of if-else conditions.
5. Missing test cases for grade boundaries.

Root cause: Incorrect comparison operators or missing boundary tests can produce wrong grades.

Correction: Validate the input range and check grade conditions in the correct order.

b) Equivalence Class Partitioning and Boundary Value Analysis

Assume the grading rules are:

Mark Range	Expected Grade
Below 0	Invalid
0–49	Fail
50–74	C
75–89	B
90–100	A
Above 100	Invalid

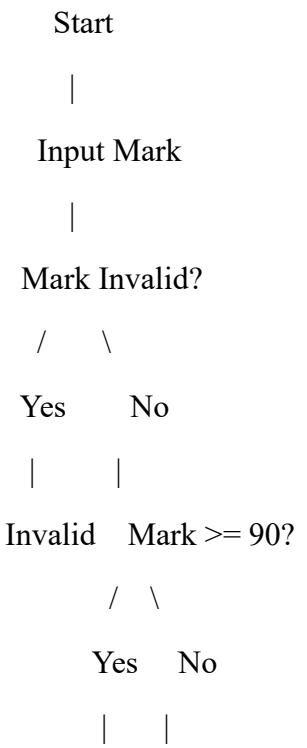
Boundary Test Cases

Test ID	Mark	Expected Output
TC01	-1	Invalid
TC02	0	Fail
TC03	49	Fail
TC04	50	C
TC05	74	C
TC06	75	B
TC07	89	B
TC08	90	A
TC09	100	A
TC10	101	Invalid

These tests help detect incorrect grade assignments near the boundaries.

c) Control Flow Graph and Cyclomatic Complexity

Control Flow Graph (CFG):



A Mark >= 75?

/ \

Yes No

| |

B Mark >= 50?

/ \

Yes No

| |

C Fail

Cyclomatic Complexity Formula:

$$V(G) = \text{Number of decision nodes} + 1$$

There are four decision nodes.

$$V(G) = 4 + 1 = \boxed{5}$$

Therefore, the cyclomatic complexity is **5** under this control-flow model.

d) JUnit Test Cases

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class ResultCalculatorTest {

   @Test
   void testGrades() {
      assertEquals("Invalid",
         ResultCalculator.calculateGrade(-1));
      assertEquals("Fail",
         ResultCalculator.calculateGrade(49));
      assertEquals("C",
         ResultCalculator.calculateGrade(50));
```

```
        assertEquals("B",
            ResultCalculator.calculateGrade(75));
        assertEquals("A",
            ResultCalculator.calculateGrade(90));
        assertEquals("Invalid",
            ResultCalculator.calculateGrade(101));
    }
}
```

These tests check invalid marks and important grade boundaries.

JaCoCo Steps:

1. Configure JaCoCo in the Maven project.
2. Run the JUnit test cases.
3. Generate the coverage report.
4. Check statement and branch coverage.
5. Add test cases for uncovered branches.

e) Mutation Testing and Test Adequacy

Mutation testing introduces small changes to the program to check whether tests detect them.

Example:

Original condition:

if (mark >= 50)

Modified condition:

if (mark > 50)

For mark 50, the expected grade is C. The modified condition may incorrectly classify it as Fail.

A good test suite should detect this error.

Conclusion: JUnit identifies incorrect outputs, JaCoCo measures code coverage, and mutation testing checks the effectiveness of test cases. Together, they improve the reliability of the grade calculator.

Q3. Error Analysis of an Online Banking Application

Aim

To identify and analyze fund-transfer errors using Postman, Swagger/OpenAPI, and black-box testing.

a) Error Identification and Root Cause

Error	Expected Behaviour	Possible Cause
Invalid amount accepted	Transaction rejected	Missing amount validation
Incorrect error message	Clear and consistent error displayed	Inconsistent error handling
Incorrect transaction status	Status reflects the actual transaction result	Improper state management
Duplicate transfer	Repeated request should not create unintended duplicate payments	Missing duplicate-request protection
Balance reduced after failure	Balance should remain unchanged	Incorrect transaction rollback
Unauthenticated transfer	Request rejected	Missing authentication check

Correction: Validate transfer requests, use consistent error responses, and maintain reliable transaction status and balance updates.

b) Black-Box Testing Techniques

1. Equivalence Class Partitioning

Input	Expected Result
Negative amount	Rejected
Zero amount	Rejected
Valid amount within balance	Accepted
Amount greater than balance	Rejected
Invalid account number	Rejected

2. Boundary Value Analysis

Assume the available balance is ₹10,000 and the daily limit is ₹20,000.

Transfer Amount	Expected Result
₹0	Rejected
₹1	Accepted if all other conditions are satisfied
₹9,999	Accepted if all other conditions are satisfied
₹10,000	Accepted if the full balance may be transferred
₹10,001	Rejected
₹20,001	Rejected

3. Random Testing

Generate random transfer amounts, invalid account numbers, missing parameters, and incorrect data types.

c) Decision Table and State Transition Testing

Authenticated	Sufficient Balance	Within Daily Limit	Result
Yes	Yes	Yes	Transfer allowed
No	Yes	Yes	Rejected
Yes	No	Yes	Rejected
Yes	Yes	No	Rejected

Transaction State Diagram

Pending

|

+----> Successful

|

+----> Failed

|

+----> Cancelled

Test network failures during transfers and verify the final status and account balance. Repeat requests to check for duplicate transactions.

Pairwise Testing: Test combinations of browser, network type, authentication state, and request repetition to identify interaction errors.

d) Postman Testing

Steps:

1. Open Postman and select the authorized test API.
2. Enter the endpoint and HTTP method.
3. Provide authentication details and request data.
4. Send valid and invalid transfer requests.
5. Record the HTTP status code and response body.
6. Compare actual results with expected results.
7. Save screenshots and report defects.

Example Test Case

Field	**Details**
Test ID	TC01
Input	Transfer ₹5,000 from an account with ₹10,000
Expected Result	Transfer successful if other rules are satisfied
Actual Result	Record after execution
Status Code	Record the observed response code

e) Test Adequacy Evaluation

Requirement traceability ensures that every transaction rule has corresponding test cases.

High-priority tests include:

- Invalid transfer amounts.
- Insufficient balance.
- Missing authentication.
- Network failures.
- Duplicate transfer requests.
- Incorrect transaction status.

When source code is unavailable, internal statement coverage, branch coverage, and cyclomatic complexity cannot normally be measured directly.

Conclusion: Postman and black-box testing help identify API validation, transaction, and network-related errors. Testing should be performed only on an authorized test environment.

Q4. Error Analysis of a COTS Learning Management System

Aim

To identify functional, compatibility, and performance errors in a third-party Learning Management System using Selenium and Apache JMeter.

a) Error Identification

Error	Expected Behaviour	Possible Cause
Login failure	Valid credentials should work	Authentication issue
Course registration failure	Eligible students should enroll	Validation or server error
Assignment upload failure	Supported files should upload	File-size or format restriction
Browser compatibility issue	Supported browsers should work	Browser-specific behaviour
Slow response	Pages should respond within the accepted time	Server overload
Incorrect results	Correct student results should be displayed	Data retrieval or processing error

b) Black-Box Testing

Equivalence Class Partitioning

For assignment uploads, test:

- Supported file types.
- Unsupported file types.
- Files within the permitted size.
- Files exceeding the permitted size.

Boundary Value Analysis

If the maximum file size is 10 MB:

File Size	Expected Result
9 MB	Accepted
9.9 MB	Accepted
10 MB	Accepted if the limit is inclusive
10.1 MB	Rejected

Exploratory Testing: Navigate through the LMS and try different workflows to discover unexpected errors.

Error Guessing: Test empty forms, interrupted uploads, invalid filenames, repeated submissions, and browser refreshes.

c) Selenium and JMeter Testing

Selenium Steps:

1. Open the LMS.
2. Test login with valid and invalid credentials.
3. Enroll in a course.
4. Upload an assignment.
5. View examination results.
6. Record errors and capture screenshots.

JMeter Steps:

1. Create a Test Plan.
2. Configure a Thread Group.
3. Set the number of virtual users.
4. Add HTTP requests for authorized test endpoints.
5. Run the test.
6. Measure response time, throughput, and error rate.
7. Compare results with the predefined acceptance criteria.

d) Limitations of COTS Testing

COTS applications usually do not provide source-code access to customers.

Therefore:

1. Internal Control Flow Graphs cannot normally be constructed.

2. Independent execution paths cannot be directly identified.
3. Source-code statement and branch coverage cannot normally be measured.
4. Testing must focus on observable behaviour, interfaces, documentation, and acceptance criteria.

e) Test Adequacy Evaluation

Evaluate the application based on:

- Functional test coverage.
- Browser and device compatibility.
- Performance under expected workloads.
- Error handling and usability.
- Security and access control.

If important tests fail, the application should not be accepted until the defects are resolved or formally assessed against the acceptance criteria.

Conclusion: Selenium identifies functional and compatibility errors, while JMeter helps detect performance problems. The final acceptance decision depends on actual results and predefined requirements.

Q5. Error Analysis of a Student Result-Processing Program

Aim

To identify logical errors in a Python result-processing program and improve its correctness using pytest, Coverage.py, and mutation testing.

a) Error Identification

Common errors include:

1. Incorrect loop execution.
2. Improper initialization of total marks.
3. Acceptance of marks outside the valid range.
4. Incorrect pass/fail conditions.
5. Incorrect subject count.
6. Reuse of old values between students.

Root Cause: Incorrect variable initialization, loop boundaries, and comparison operators may produce incorrect results.

Correction: Initialize totals for each student, validate marks, and use correct loop and pass/fail conditions.

b) Test Cases

Assume marks range from 0 to 100 and the pass mark is 50.

Test ID	Input	Expected Result
TC01	Mark = -1	Invalid
TC02	Mark = 0	Fail
TC03	Mark = 49	Fail
TC04	Mark = 50	Pass
TC05	Mark = 100	Pass
TC06	Mark = 101	Invalid
TC07	Student count = 0	Validation message or empty result
TC08	Subject count = 0	Validation message
TC09	Multiple students with different marks	Each result calculated correctly

Test ID	Input	Expected Result
TC10	Missing or non-numeric mark	Input rejected

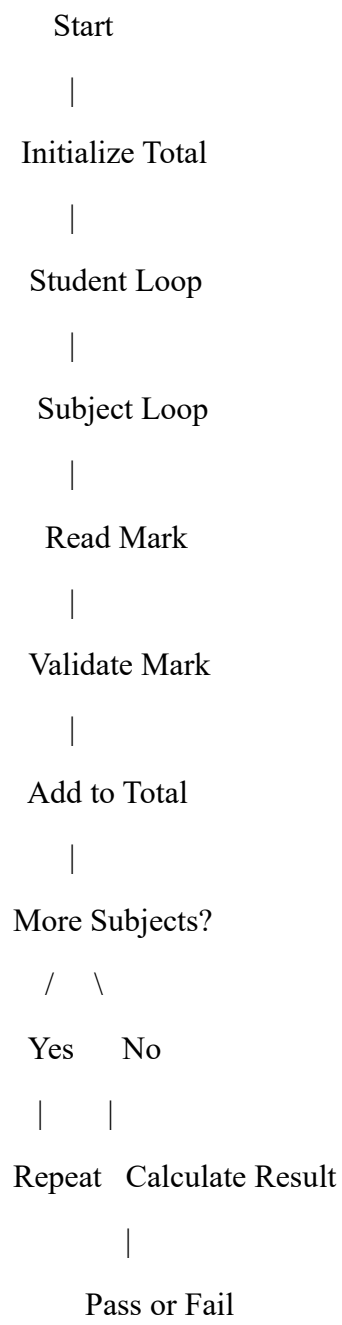
Equivalence Class Partitioning: Divide marks into invalid, failing, and passing groups.

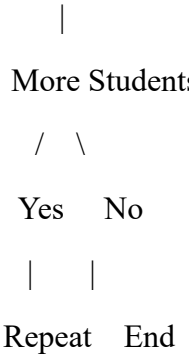
Boundary Value Analysis: Test marks around 0, 50, and 100.

Error Guessing: Test zero students, negative counts, missing marks, and incorrect data types.

c) Control Flow Graph and Cyclomatic Complexity

Example CFG





Cyclomatic Complexity Formula:

$$V(G) = \text{Number of decision nodes} + 1$$

For this example, assume there are four decision nodes: the student loop, subject loop, mark validation, and pass/fail decision.

$$V(G) = 4 + 1 = \boxed{5}$$

Therefore, the estimated cyclomatic complexity is **5**, based on the illustrated control-flow model. The actual value depends on the program's source code.

d) Python Program and pytest

Save the program as result.py.

```
def calculate_result(marks):
    if not marks:
        return "Invalid"

    total = 0

    for mark in marks:
        if mark < 0 or mark > 100:
            return "Invalid"
        total += mark

    average = total / len(marks)
```

```
    if average >= 50:
        return "Pass"
    return "Fail"
```

Create test_result.py:

```
from result import calculate_result

def test_pass():
    assert calculate_result([60, 70]) == "Pass"

def test_fail():
    assert calculate_result([30, 40]) == "Fail"

def test_invalid_mark():
    assert calculate_result([50, 101]) == "Invalid"

def test_empty_marks():
    assert calculate_result([]) == "Invalid"

def test_boundary():
    assert calculate_result([50]) == "Pass"
```

Run the tests:

Install the tools:

```
pip install pytest coverage
```

Execute the test suite:

```
pytest -v
```

Measure statement coverage:

```
coverage run -m pytest
coverage report -m
```

Measure branch coverage:

coverage run --branch -m pytest

coverage report -m

e) Data-Flow and Mutation Testing

Data-flow testing checks where variables are initialized, modified, and used.

Example: If total is not reset for each student, the next student's result may include the previous student's marks.

Mutation testing changes conditions to check whether the test suite detects errors.

Original condition:

if average >= 50:

Modified condition:

if average > 50:

For an average of exactly 50, the original program returns Pass, but the modified program returns Fail. The boundary test should detect this error.

f) Test Adequacy Evaluation

Evaluate the final test suite using:

- **Statement coverage:** Statements executed.
- **Branch coverage:** True and false decision outcomes tested.
- **Path testing:** Important execution paths checked.
- **Data-flow testing:** Variable initialization and use checked.
- **Mutation testing:** Incorrect logic changes detected.

High coverage alone does not guarantee correctness. Additional tests should check multiple students, multiple subjects, invalid inputs, and loop boundaries.

Conclusion: pytest and Coverage.py help identify untested code, while data-flow and mutation testing detect initialization and logical errors. The program should be released only after important defects are corrected and the test results satisfy the required criteria.